

Report - Best Histogram Equalization method

Berno Sara, Carretta Francesco, Piccoli Leonardo Arduino

1 Introduction and code structure

The goal of this project is to implement, test, and compare different histogram equalization techniques for grayscale images, using C++ with OpenCV, in order to find the best technique to improve original images. We evaluate both quantitative and qualitative performance on a provided dataset, applying multiple metrics (PSNR, SSIM, AMBE, Entropy, Standard Deviation, BRISQUE). The experiments are carried out on the provided dataset, and both quantitative and qualitative results are reported for each image. Results show that no single method is optimal in all scenarios.

Our system tries to comply with all the standard requirements, achieving in this way better portability and readability. To maintain a good degree of scalability, we tried our best to maintain a reasonably low level of complexity, avoiding reimplementing from scratch functions that were already defined in the libraries (which already guarantee optimal performance) and trying to optimize every newly defined method.

Project Structure

For enhanced readability, the project is organized into 3+1 main folders:

- **data:** the initial dataset, with all the images subjected to future enhancement.
- **include:** header files (`methods.hpp`, `params.hpp`, `metrics.hpp`, `metricsList.hpp`, `utilities.hpp`).
- **src:** source files (`main.cpp`, `equalizations.cpp`, `metrics.cpp`, `utilities.cpp`).
- **CMakeLists.txt:** CMake configuration file.
- **output:** contains the modified images after the execution of the program. It is organized into K subdirectories (one for each of the K images of the initial dataset, 31 in our test case). Each subfolder contains copies of the K -th image obtained after the execution of each histogram equalization technique. A CSV file reporting the obtained numerical results of the chosen evaluation metrics is therefore generated.

Libraries

In the development of the program, we used only the C++ standard library and the following OpenCV modules:

- `<opencv2/imgproc.hpp>` – for image processing (histogram equalization, histogram calculation, min/max search, mean/stddev, LUT operations, etc.).
- `<opencv2/imgcodecs.hpp>` – for image loading and saving (`cv::imread`, `cv::imwrite`).

- `<opencv2/quality.hpp>` – for image quality metrics (PSNR, SSIM, BRISQUE...).
- `<opencv2/core.hpp>`

File organization by author

Berno Sara

main.cpp Entry point of the program. It manages input/output directories, scans the dataset for images, applies all equalization methods, computes the metrics, saves results (images, histograms, and CSV), and finally identifies the best method for each image.

utilities.hpp / utilities.cpp Provide helper functions:

- **applyEqualization**: calls the appropriate equalization method;
- **computeMetrics**: calculates all evaluation metrics for an image;
- **listImages**: recursively scans directories to collect input images;
- **renderHistogramImage**: generates a histogram plot as an image;
- **findTheBest**: compares all results based on metrics and saves the best equalization for each image into the **BEST/** folder.

Carretta Francesco

metrics.hpp / metrics.cpp Define and implement the performance metrics used to evaluate equalized images against the original input. Metrics include:

- Mean brightness
- AMBE (Absolute Mean Brightness Error)
- PSNR (Peak Signal-to-Noise Ratio)
- SSIM (Structural Similarity Index)
- Entropy
- Standard Deviation
- BRISQUE (Blind/Referenceless Image Spatial Quality Evaluator)
(a more in-depth description can be found in paragraph 4).

metricsList.hpp Defines the `bhe::MetricsList` struct, a container for all computed metrics. It allows passing results between functions and saving them into CSV files.

Piccoli Arduino Leonardo

equalization.hpp / equalizations.cpp Define and implement all histogram equalization techniques requested by the proposal:

- Global Histogram Equalization (HE)
- CLAHE (Contrast Limited Adaptive HE)
- Brightness Preserving Bi-Histogram Equalization (BI-HE)
- Multi-Histogram Equalization (Multi-HE)
- Histogram Matching / Specification

Helper functions for histogram CDF calculation, segmentation, and LUT building are also provided (a more in-depth description can be found in paragraph 4).

`methods.hpp` Defines the `bhe::Methods` enumeration, which lists all available equalization techniques and enables their selection in the program.

`params.hpp` Defines the `bhe::Params` structure, which contains configurable parameters for the algorithms (e.g., CLAHE clip limit, grid size, Multi-HE partitions, target Gaussian distribution for Matching-HE).

Contributions and workload

- Berno Sara: implemented `main.cpp` and `utilities.cpp`; managed dataset handling, output saving, and “find the best” logic; implemented entropy metric and BI-HE; wrote the report; performed experiments. 30 hours.
- Carretta Francesco: implemented `metrics.cpp`; integrated BRISQUE, PSNR, SSIM and Multi-HE; performed experiments, collected metric results and implemented helper functions (CDF, LUT); wrote the report. 30 hours.
- Piccoli Leonardo Arduino: implemented `equalizations.cpp` and `utilities.cpp`; designed and coded equalization techniques (Global HE, CLAHE, Matching), organized files paths and managed the CMake procedure; wrote the report. 30 hours.

2 Implemented equalization methods

Following the indications of the project proposal guideline, we implemented the following histogram equalization techniques:

2.1 Global Histogram Equalization (HE)

```
cv::Mat bhe::applyGlobalHE(const cv::Mat& gray_img)
```

This is the standard OpenCV implementation (`cv::equalizeHist`). It redistributes the pixel intensities so that the cumulative distribution function (CDF) becomes nearly linear. The method enhances global contrast especially when illumination is poor. However, it may shift the overall brightness and can amplify noise in very dark or bright regions.

2.2 Contrast Limited Adaptive Histogram Equalization (CLAHE)

```
cv::Mat bhe::applyCLAHE(const cv::Mat& gray_img, double  
    clip_limit, cv::Size tile_grid)
```

This method performs histogram equalization locally, dividing the image into tiles that are processed independently and then combined using bilinear interpolation. The parameter `clip_limit` avoids excessive contrast amplification caused by noise. `tile_grid` controls the local neighborhood. This approach enhances local contrast and produces more natural results than global equalization, but it involves the use of parameter that need to be tuned and is computationally more expensive.

2.3 Brightness Preserving Bi-Histogram Equalization (BI_HE)

```
cv::Mat bhe::applyBIHE(const cv::Mat& gray_img)
```

In this method, the histogram is split into two parts using the mean intensity T of the image as a threshold. Each sub-histogram is equalized independently and then merged. By equalizing around the average brightness, the method should preserve global luminance better if compared to global HE. It prevents extreme brightness shifts, although it is less adaptive when the intensity distribution is highly non-uniform.

2.4 Brightness Preserving Multi-Histogram Equalization (Multi_HE)

```
cv::Mat bhe::applyMultiHE(const cv::Mat& gray_img, int
    multi_parts)
```

This function generalizes BI_HE by splitting the histogram into N sub-histograms, with the cut points determined automatically from the CDF so that each segment contains a similar fraction of pixels. Each part is equalized separately and then recombined. This technique should provide finer control over contrast enhancement across different intensity ranges and allows a balance between brightness preservation and contrast enhancement. It is needed to choose the number of partitions and the additional computational cost.

2.5 Histogram Matching/Specification

```
cv::Mat bhe::applyMatching(const cv::Mat& gray_img, float mean,
    float variance)
```

Instead of flattening the histogram, this method shapes it to match a target distribution. In our implementation, the target is an inverted Gaussian distribution defined by a mean and a variance. The input CDF is matched against the target CDF, and a lookup table is used to remap pixel intensities accordingly. This provides more controlled enhancement by forcing the output histogram towards a desired form. The effectiveness depends strongly on the choice of the target distribution: if poorly chosen, the results may look unnatural.

Helper functions such as CDF computation, histogram partitioning, and lookup table generation are shared across these methods, improving modularity and reusability.

3 Evaluation metrics

To assess the quality of the equalized images, we implemented a set of metrics defined in `metrics.hpp` and `metrics.cpp`. All metrics assume 8-bit grayscale images (CV_8UC1). Some metrics are reference-based, comparing the processed image with the original (AMBE, PSNR, SSIM), while others are quality indicators. The results for each image and method are written into a CSV file with the following format:

```
file, method, brightness, ambe, psnr, ssim, entropy, std_deviation, brisque
```

To select the best method for each image, all metrics are first min-max normalized across the candidate methods of that image. Metrics where lower values are preferable (AMBE, BRISQUE) are inverted after normalization. A weighted aggregate score is then computed:

$$\text{Score} = 0.30 \cdot \text{SSIM}_N + 0.25 \cdot (1 - \text{BRISQUE}_N) + 0.20 \cdot \text{PSNR}_N + 0.15 \cdot (1 - \text{AMBE}_N) + 0.10 \cdot \text{Entropy}_N.$$

The method with the highest score is selected as the winner for that image.

3.1 Mean Brightness

```
double bhe::meanIntensity(const cv::Mat& processed_img)
```

Computes the average gray level of the processed image, giving a measure of global luminance.

3.2 AMBE (Absolute Mean Brightness Error)

```
double bhe::ambe(const cv::Mat& reference_img, const cv::Mat&
    processed_img)
```

Quantifies brightness preservation by computing the absolute difference between the mean intensity of the processed image and that of the original. Lower values indicate better preservation.

3.3 PSNR (Peak Signal-to-Noise Ratio)

```
double bhe::psnr(const cv::Mat& reference_img, const cv::Mat&
    processed_img)
```

Measures the fidelity of the processed image relative to the reference. Implemented with the OpenCV Quality module:

```
cv::quality::QualityPSNR::compute(reference, processed, cv::
    noArray(), 255.0)[0];
```

Higher values indicate greater similarity.

3.4 SSIM (Structural Similarity Index)

```
double bhe::ssim(const cv::Mat& reference_img, const cv::Mat&
    processed_img)
```

Evaluates perceptual similarity in terms of luminance, contrast, and structure, using the OpenCV Quality module:

```
cv::quality::QualitySSIM::compute(reference, processed, cv::
    noArray()) [0];
```

Its range is $[0, 1]$, with values closer to 1 indicating higher similarity.

3.5 Shannon Entropy

```
double bhe::entropy(const cv::Mat& processed_img)
```

Calculates the Shannon entropy of the gray-level distribution. It is obtained by building a 256-bin histogram, normalizing it into probabilities p_i , and computing

$$H = - \sum_i p_i \log_2(p_i) \quad (p_i > 0).$$

Higher entropy often corresponds to richer contrast and detail, though it may also reflect noise.

3.6 Standard Deviation

```
double bhe::stdDeviation(const cv::Mat& processed_img)
```

Measures global contrast spread as the standard deviation of pixel intensities. Implemented with:

```
cv::meanStdDev(img, mean, stddev);  
return stddev[0];
```

Larger values indicate stronger contrast.

3.7 BRISQUE (Blind/Referenceless Image Spatial Quality Evaluator)

```
double bhe::brisque(const cv::Mat& processed_img)
```

A no-reference perceptual metric based on natural scene statistics, implemented with the OpenCV Quality module. It requires model files provided by OpenCV:

- /usr/local/share/opencv4/quality/brisque_model_live.yml
- /usr/local/share/opencv4/quality/brisque_range_live.yml

The function caches a `cv::quality::QualityBRISQUE` instance and returns:

```
quality_b->compute(processed_img)[0];
```

Lower scores indicate better perceptual quality (closer to natural-looking images).

In summary, the combination of reference-based metrics (AMBE, PSNR, SSIM) and perceptual measures (Entropy, Std. Dev, BRISQUE) allows a balanced evaluation of the results, taking into account both objective fidelity and visual quality.

4 CMake and Execution

This section describes how to compile and run the project in the official environment, the Virtual Lab, used to make the process reproducible and independent of the local machine.

CMake configuration

The `CMakeLists.txt` file specifies:

1. The C++ standard (C++17) and the default build type (*Release*).
2. The search for OpenCV and the inclusion of its directories.
3. The creation of the executable from the source files in `src/`.
4. Linking against the required OpenCV libraries.

The output binaries are placed in `build/bin` in order to keep the project root clean.

Compilation procedure

The compilation follows the standard CMake workflow:

1. From the project root, create the build directory: `mkdir build`
2. Enter the build directory: `cd build`
3. Generate the project files: `cmake ..`
4. Compile: `cmake --build .`

At the end of the process, the executable `BestHistogramEqualization` will be available in `build/bin`.

Execution

The executable takes as input the dataset folder, the output folder, and optionally the folder for saving the CSV metrics file. The syntax is the following:

```
./bin/BestHistogramEqualization <input_dataset_dir> <output_root_dir>
[csv_dir]
```

so, for example,

```
./bin/BestHistogramEqualization ../data ../output_results
```

- `<input_dataset_dir>` is the folder containing the input images (subdirectories are allowed). Supported extensions are: `.png`, `.jpg`, `.jpeg`, `.bmp`, `.tif`, `.tiff`, `.webp`.
- `<output_root_dir>` is the folder where the results will be written. The program automatically creates a subfolder `output_data/`.
- `[csv_dir]` is optional: if provided, the file `metrics.csv` is written in this folder; otherwise, it is saved inside `<output_root_dir>/output_data/`.

Generated output

The execution produces:

- For each input image, a dedicated folder in `<output_root_dir>/output_data/<image_name>/`, containing:
 - The histogram of the original image (`_hist.png`).
 - A subfolder for each equalization method, with the equalized image and its histogram.
- A `metrics.csv` file with all computed metrics for every method and every image. The columns include: `file`, `method`, `brightness`, `ambe`, `psnr`, `ssim`, `entropy`, `std_deviation`, `brisque`.
- A folder `BEST/` that collects, for each image, the best result according to the ranking criterion written above, along with the corresponding histogram.

Parameters and configuration

The parameters for the algorithms (e.g. `clip_limit` and `tile_grid` for CLAHE, `multi_parts` for Multi-HE, `mean` and `variance` for Matching) are stored in the `bhe::Params` structure and instantiated in the `main` file.

5 Conclusions

The results show that no single method is universally optimal:

- **Global-HE** usually maximizes entropy and variance, producing more uniform histograms, but often generates unnatural contrast and noise amplification (e.g., images 23 and 27).
- **CLA-HE** provides the most consistent improvements: it enhances local contrast without over-amplifying noise, and in most cases obtains better perceptual scores (lower BRISQUE, higher SSIM), as visible in images 10, 16, and 19.
- **BI-HE** and **Multi-HE** preserve brightness more effectively and produce visually natural results. Multi-HE in particular showed good balance in challenging low-light scenarios (e.g., Image 26), even if its histograms are less uniform.
- **Histogram Matching** is highly dependent on the chosen target distribution: in some cases it reaches high PSNR/SSIM (Image 21), but in others it yields poor perceptual quality (e.g., high BRISQUE in image 19).

From the dataset as a whole, CLA-HE emerges as the most reliable method, achieving a good trade-off across all metrics and producing visually pleasant results. MULTI-HE and BI-HE are also competitive when brightness preservation is required, while GLOBAL-HE can be useful when maximizing global contrast is the priority, despite the risk of artifacts.

Histogram matching is better suited when a specific reference distribution is needed, that is because a high quality image’s histogram should have been used instead of a predefined function, but it was non given by the project proposal authors.

Furthermore, in the dataset provided there are images that cannot be fully improved with equalization alone, filters should be applied to the blurred or noisy samples.

Histogram equalization remains a valuable tool in image enhancement, but the “best” technique strongly depends on the application. The experiments confirm the importance of multi-metric evaluation: visual inspection alone or a single numerical index cannot fully capture the quality of enhancement. CLA-HE appears as the most versatile approach, while MULTI-HE provides an interesting alternative in difficult illumination conditions.